CYBER SECURITY INTERNSHIP PROGRAM

WEEK 6 TECHNICAL LAB REPORT

Web Security & Vulnerability Testing (VAPT Basics) — SQL Injection, Cross-Site Scripting and Automated Scanning using DVWA, Burp Suite and OWASP ZAP

Submitted By: Diksha Uma Shankar Raul

Internship Organization: DG Cyber Security Internship

Submission Deadline: 14 October 2026

Target Application: DVWA (Damn Vulnerable Web Application)

Attacker Machine: Kali Linux

Tools Used: Burp Suite Community Edition, OWASP ZAP

TABLE OF CONTENTS

Cover Page 1
Table of Contents / List of Figures 2

1. Executive Summary 3
2. Introduction to VAPT 3
   2.1 SQL Injection 3
   2.2 Cross-Site Scripting (XSS) 3
   2.3 Broken Authentication 3
   2.4 Security Misconfiguration 3
3. Tool Installation & Target Setup (Task 2) 3
4. SQL Injection Testing (Task 3) 4
5. Cross-Site Scripting Testing (Task 3 contd.) 6
   5.1 Reflected XSS 6
   5.2 Stored XSS 7
6. Automated Scanning with OWASP ZAP (Task 4) 8
7. Findings & Risk Levels 9
8. Mitigation Methods 10
9. Conclusion & Learning Outcomes 10

1. Executive Summary

This report documents the Week 6 tasks of the DG Cyber Security Internship, covering web application vulnerability assessment and penetration testing (VAPT) basics. All testing was carried out in a safe, isolated lab against the Damn Vulnerable Web Application (DVWA) running on Kali Linux, with security level set to Low.

Key activities included studying the OWASP Top 10, installing and configuring DVWA, Burp Suite and OWASP ZAP, performing manual SQL Injection and Cross-Site Scripting (Reflected and Stored) attacks while capturing traffic in Burp Suite, and running an automated vulnerability scan with OWASP ZAP. The findings, risk levels and recommended mitigations are summarised in Section 7.

2. Introduction to VAPT

Vulnerability Assessment and Penetration Testing (VAPT) is the practice of identifying, verifying and exploiting weaknesses in an application in a controlled manner, so that they can be fixed before an attacker finds them. Vulnerability Assessment focuses on discovering and listing weaknesses, while Penetration Testing goes a step further and actively exploits them to prove real-world impact.

The OWASP Top 10 is an industry-standard list of the most critical web application security risks. For this task, four categories were studied in depth: SQL Injection, Cross-Site Scripting (XSS), Broken Authentication, and Security Misconfiguration.

2.1 SQL Injection

SQL Injection is a code injection vulnerability that occurs when an application builds a SQL query by directly concatenating untrusted user input into the query string, instead of treating that input as pure data. Because the database cannot tell the difference between "code" and "data" once they're merged into one string, an attacker can insert SQL syntax (quotes, operators, keywords like `OR`, `UNION`, `--`) that changes the meaning and structure of the original query.

2.2 Cross-Site Scripting (XSS)

XSS is a client-side injection vulnerability where an application includes untrusted input in a web page's output without proper encoding or sanitisation, allowing an attacker's JavaScript to execute inside another user's browser session, under the trust context (cookies, DOM, session) of the vulnerable site.

2.3 Broken Authentication

Broken Authentication refers to a class of weaknesses in how an application confirms and maintains a user's identity — covering login, session management, password handling, and credential recovery — that allow an attacker to assume another user's identity, either by guessing/stealing credentials or by exploiting flaws in how sessions are issued and validated.

2.4 Security Misconfiguration

Security Misconfiguration is a broad category covering any point where an application, server, framework, or platform is deployed with insecure settings — whether through unsafe defaults, incomplete configuration, overly verbose error handling, or unnecessary features left enabled — that don't directly breach the system on their own but weaken its defenses and leak information that helps other attacks succeed.

3. Tool Installation & Target Setup (Task 2)

DVWA was installed on Kali Linux and started locally. Burp Suite Community Edition and OWASP ZAP are pre-installed on Kali and were used for intercepting and scanning traffic respectively. After logging into DVWA

(admin / password), the DVWA Security level was set to Low so that vulnerabilities could be exploited without additional filtering, as required for this learning exercise.

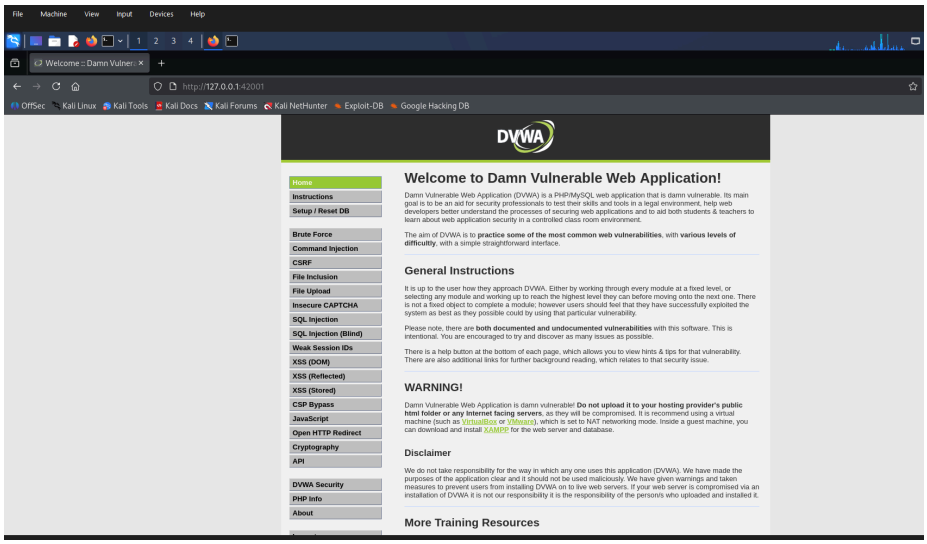


Figure 3.1: DVWA home page after successful login.

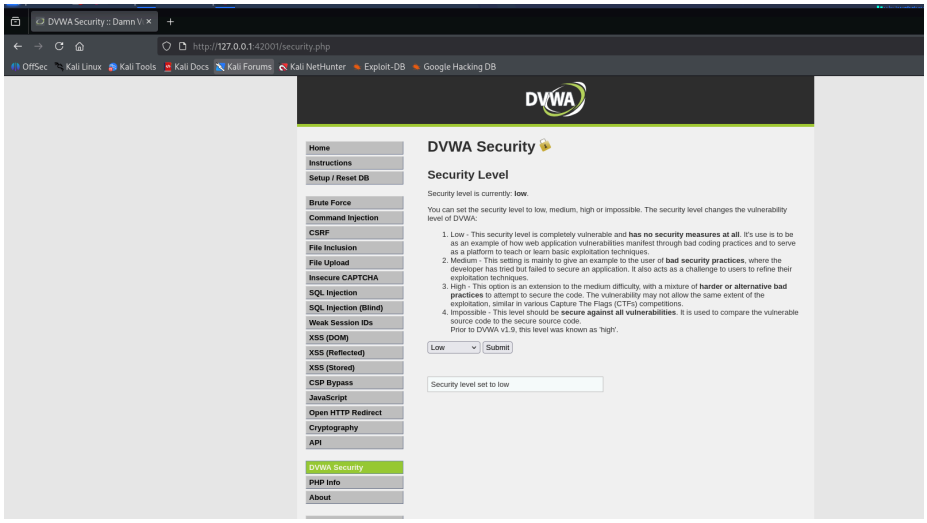


Figure 3.2: DVWA Security level set to Low.

4. SQL Injection Testing (Task 3)

Burp Suite was configured as an intercepting proxy between the browser and DVWA, and traffic was captured while testing the SQL Injection module.

```
Payload used:  1' OR '1'='1
```

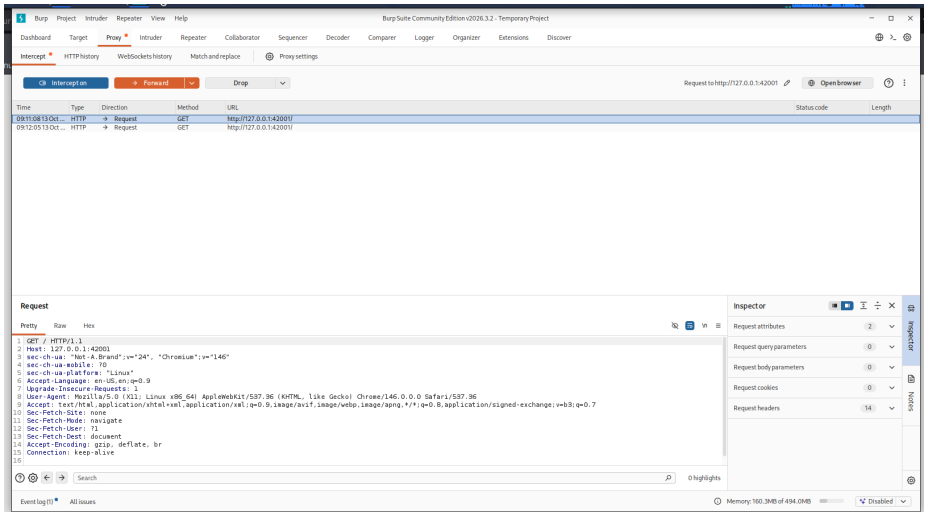


Figure 4.1: Burp Suite intercepting a request to DVWA.

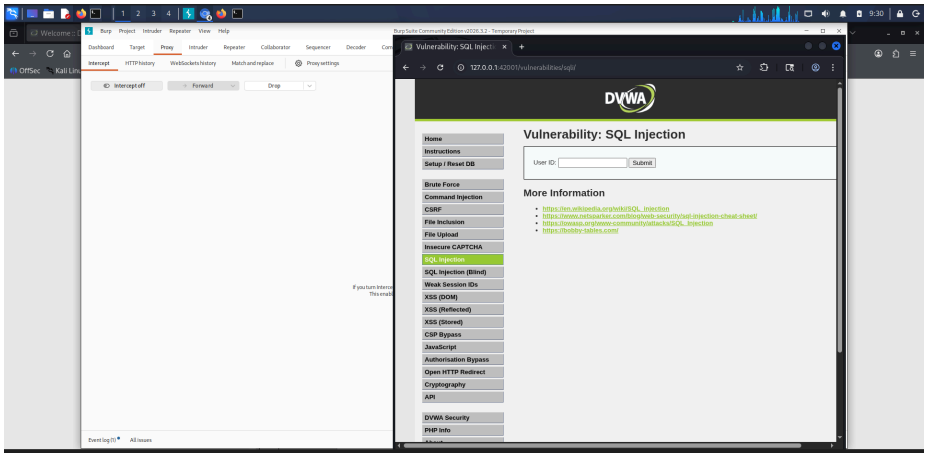


Figure 4.2: DVWA SQL Injection module — User ID input form.

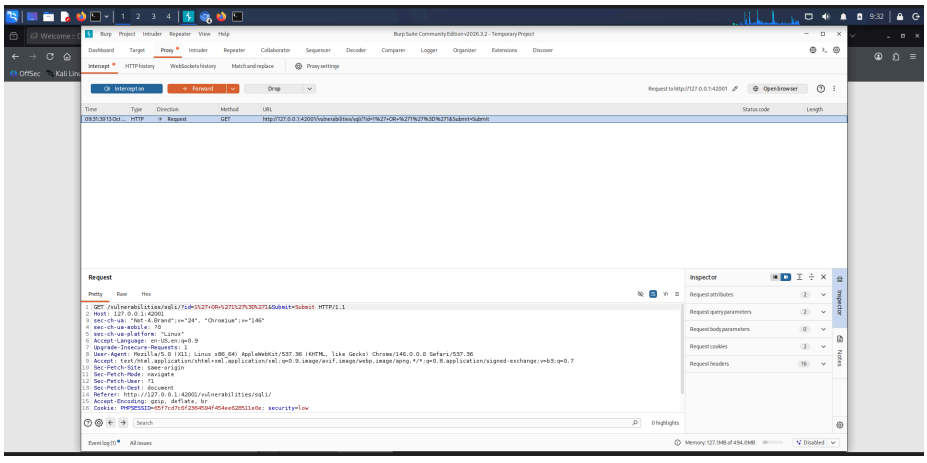


Figure 4.3: SQL Injection payload captured in Burp Suite Proxy (GET request with 1' OR '1'='1).

Submitting the payload returned every row in the users table instead of a single user, confirming that user-supplied input was concatenated directly into the backend SQL query without sanitisation.

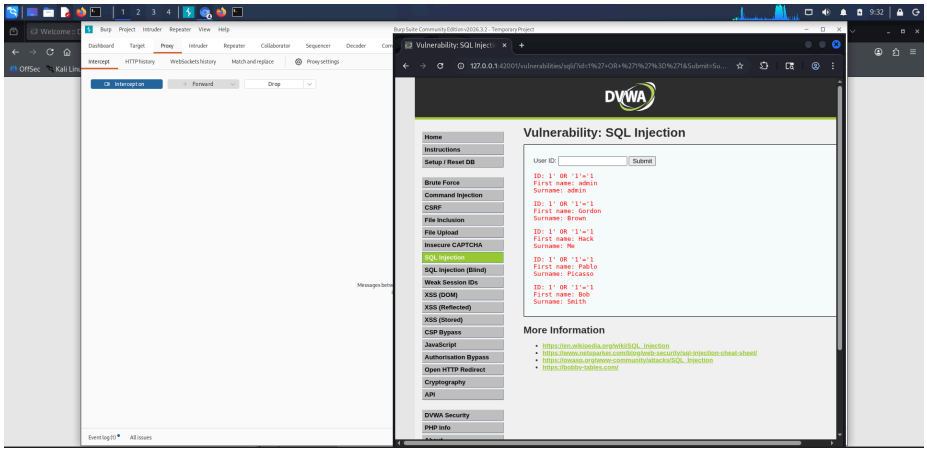


Figure 4.4: Application output showing all user records returned by the injected query.

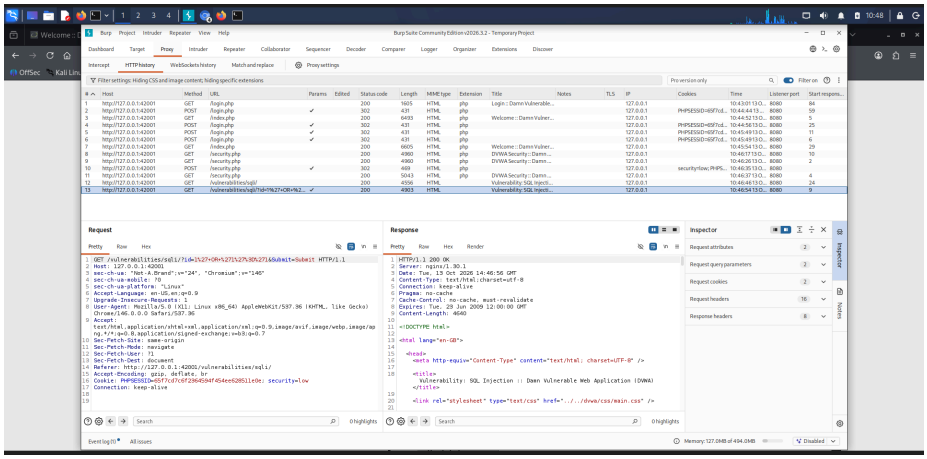


Figure 4.5: Burp Suite HTTP history — SQL Injection request and corresponding 200 OK response.

5. Cross-Site Scripting Testing (Task 3 contd.)

5.1 Reflected XSS

```
Payload used:  <script>alert('XSS')</script>
```

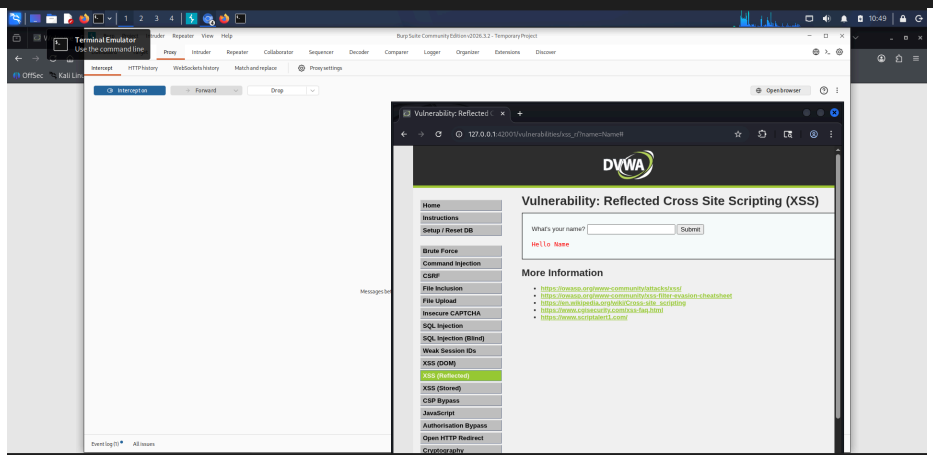


Figure 5.1: DVWA Reflected XSS module.

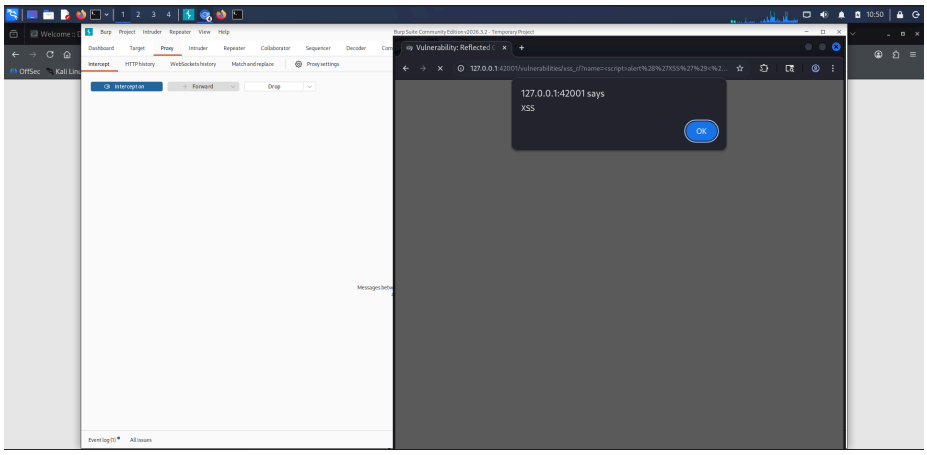


Figure 5.2: JavaScript alert box confirming successful Reflected XSS execution.

5.2 Stored XSS

```
Payload used:  <script>alert('Stored XSS')</script>
```

The payload was submitted through the DVWA guestbook (Name/Message) form. The alert fired immediately on submission, and fired again automatically on page refresh with no further input, confirming that the script was permanently stored in the database rather than merely reflected.

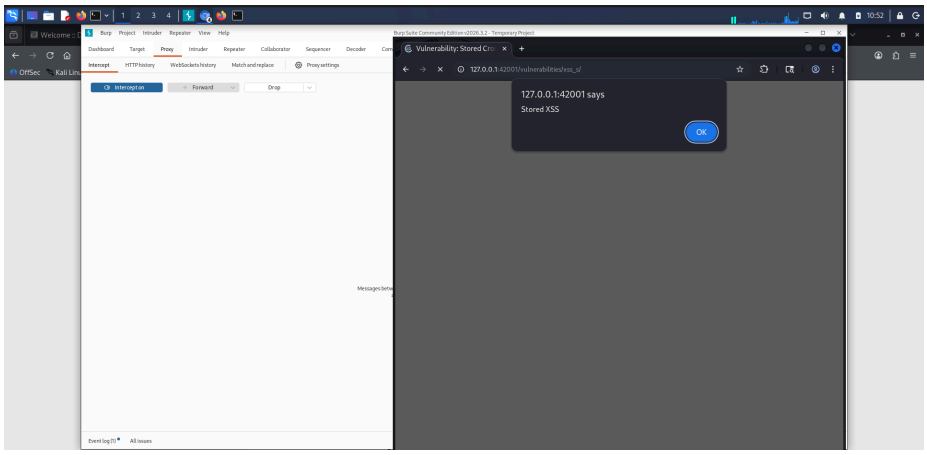


Figure 5.3: Alert box confirming the Stored XSS payload executed on submission.

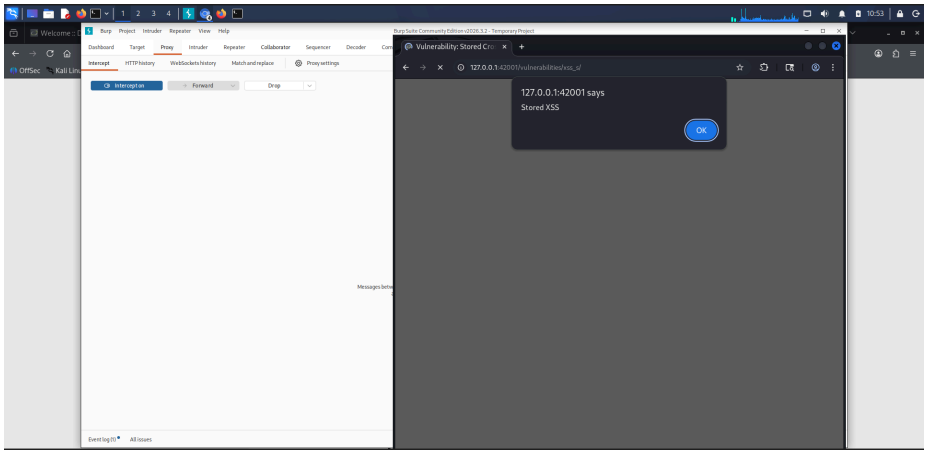


Figure 5.4: Alert box reappearing automatically after page refresh, confirming persistence.

6. Automated Scanning with OWASP ZAP (Task 4)

An Automated Scan was launched in OWASP ZAP against the DVWA target to passively and actively identify common misconfigurations and vulnerabilities, complementing the manual testing above.

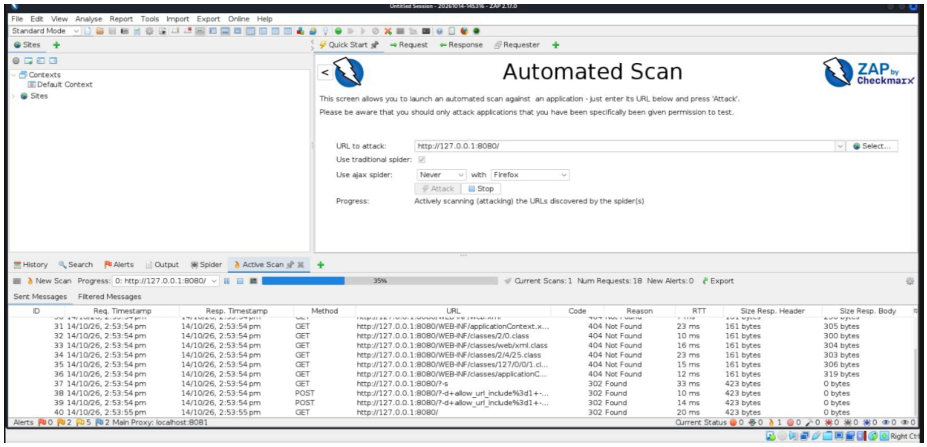


Figure 6.1: OWASP ZAP Active Scan in progress against the target.

The scan generated 9 distinct alerts (10 instances total). Two representative findings are shown below:

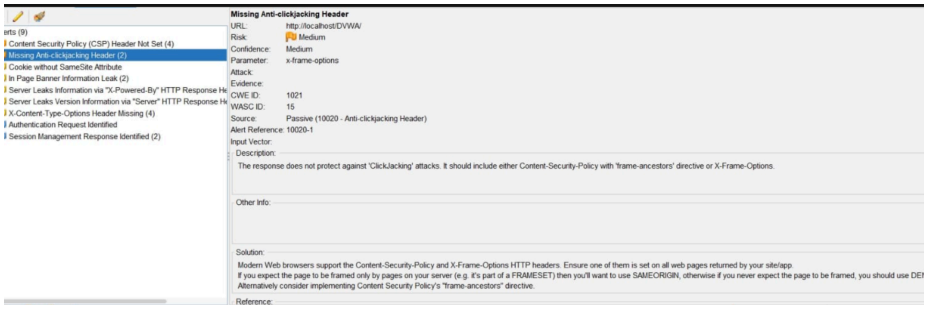


Figure 6.2: ZAP alert — Missing Anti-Clickjacking Header (Medium risk).

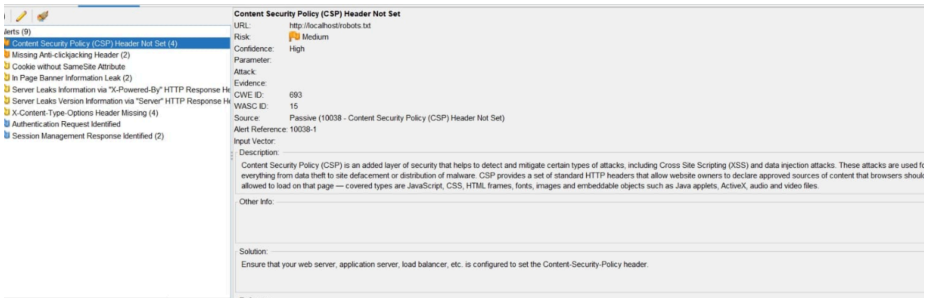
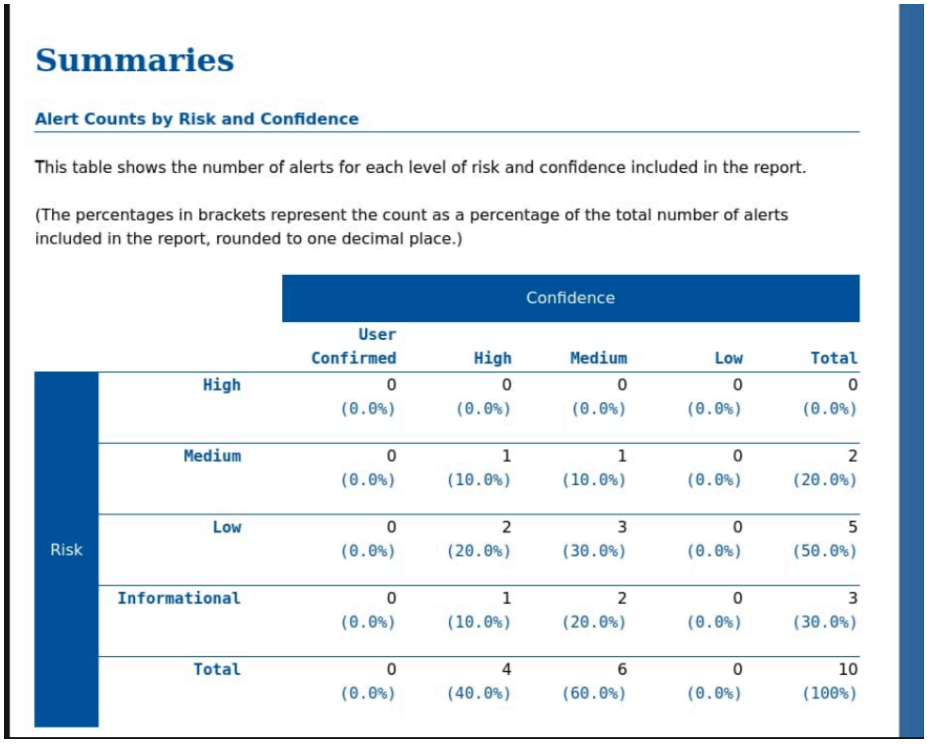


Figure 6.3: ZAP alert — Content Security Policy (CSP) Header Not Set (Medium risk).

A summary report was exported directly from ZAP for inclusion in this submission (Task 4 deliverable):



Summaries

Alert Counts by Risk and Confidence

This table shows the number of alerts for each level of risk and confidence included in the report.

(The percentages in brackets represent the count as a percentage of the total number of alerts included in the report, rounded to one decimal place.)

		Confidence				
		User Confirmed	High	Medium	Low	Total
Risk	High	0 (0.0%)	0 (0.0%)	0 (0.0%)	0 (0.0%)	0 (0.0%)
	Medium	0 (0.0%)	1 (10.0%)	1 (10.0%)	0 (0.0%)	2 (20.0%)
	Low	0 (0.0%)	2 (20.0%)	3 (30.0%)	0 (0.0%)	5 (50.0%)
	Informational	0 (0.0%)	1 (10.0%)	2 (20.0%)	0 (0.0%)	3 (30.0%)
	Total	0 (0.0%)	4 (40.0%)	6 (60.0%)	0 (0.0%)	10 (100%)

Figure 6.4: Exported ZAP report — Alert Counts by Risk and Confidence.

7. Findings & Risk Levels

Vulnerability	Risk Level	Tool Used	Impact
SQL Injection (sqli, Low security)	High	DVWA + Burp Suite	Full user table dumped via 1' OR '1'='1 payload
Reflected XSS	Medium	DVWA (xss_r)	Arbitrary script executes from URL/input, not stored
Stored XSS	High	DVWA (xss_s)	Script persists in DB, executes on every page load for every visitor

Missing Anti-Clickjacking Header	Medium	OWASP ZAP	Page can be framed, enabling clickjacking attacks
CSP Header Not Set	Medium	OWASP ZAP	No browser-level mitigation against XSS / data injection
Cookie without SameSite / Info Leak headers	Low	OWASP ZAP	Minor information disclosure, CSRF exposure

8. Mitigation Methods

- SQL Injection: Use parameterised queries / prepared statements instead of string concatenation; apply strict input validation and least-privilege database accounts.
- Reflected & Stored XSS: Encode all user-supplied output (HTML-entity encode), validate input on the server side, and implement a Content-Security-Policy header to restrict script execution.
- Missing Anti-Clickjacking Header: Set the X-Frame-Options header (e.g. SAMEORIGIN) or a CSP frame-ancestors directive on every response.
- Missing CSP Header: Define and enforce a Content-Security-Policy header restricting allowed script, style and frame sources.
- General: Keep DVWA security level and similar debug/training configurations out of production; apply the same header and input-validation hardening to real applications.

9. Conclusion & Learning Outcomes

This lab provided hands-on experience with the core VAPT workflow: studying the theory behind common web vulnerabilities (OWASP Top 10), setting up a safe testing environment with DVWA, manually exploiting SQL Injection and both forms of XSS while capturing traffic in Burp Suite, and running an automated scan with OWASP ZAP to catch configuration-level weaknesses. Together, these exercises demonstrate how manual testing and automated scanning complement each other in a complete security assessment, and reinforce why input validation, output encoding and secure HTTP headers are essential defences for any production web application.

Report Prepared By:
Diksha Uma Shankar Raul

DG Cyber Security Internship — Week 6, October 2026